CEFET-MG | DEPARTAMENTO DE COMPUTAÇÃO

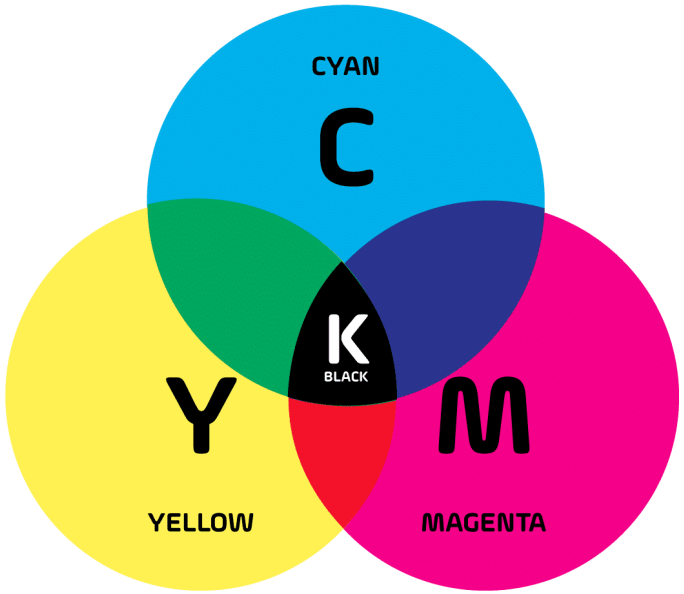
Computação Gráfica

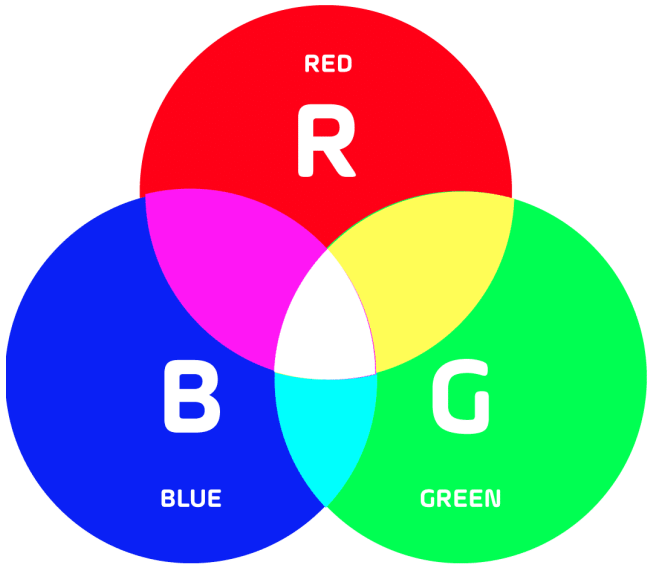
Fundamentos de Cor

Prof. Dr. Raulivan Rodrigo

Contextualização

Na aula anterior, entendemos como as máquinas representam cores através do RGB e CMYK.





Contextualização

Porém, surge um problema

Definir a cor "Marrom Claro" misturando intensidades de luz Vermelha, Verde e Azul.

O cérebro humano não processa cores de forma aditiva.

Nós percebemos a cor base, a sua pureza e a sua iluminação.



Contextualização

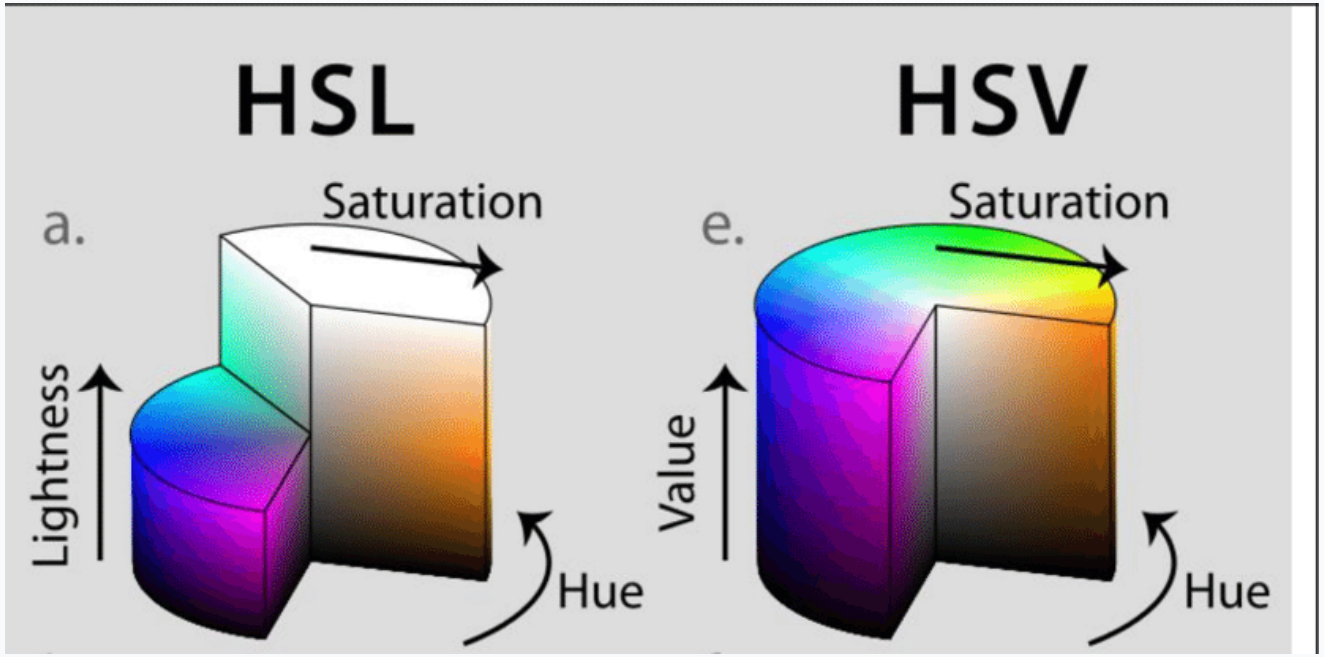
O **RGB** modela como um monitor emite luz.
Já os modelos **HSV** (Hue, Saturation, Value) e **HSL** (Hue, Saturation, Lightness) modelam como o olho e o cérebro humano organizam e interpretam essa luz.

Eles foram criados na década de 1970 por pesquisadores de computação gráfica para facilitar a seleção de cores por seres humanos.

Fundamentos de Cor

Geometria Cilíndrica: HSV e HSL

- Tanto o HSV quanto o HSL são representações matemáticas do espaço RGB, mapeadas de um cubo (vetores R, G, B) para coordenadas cilíndricas.



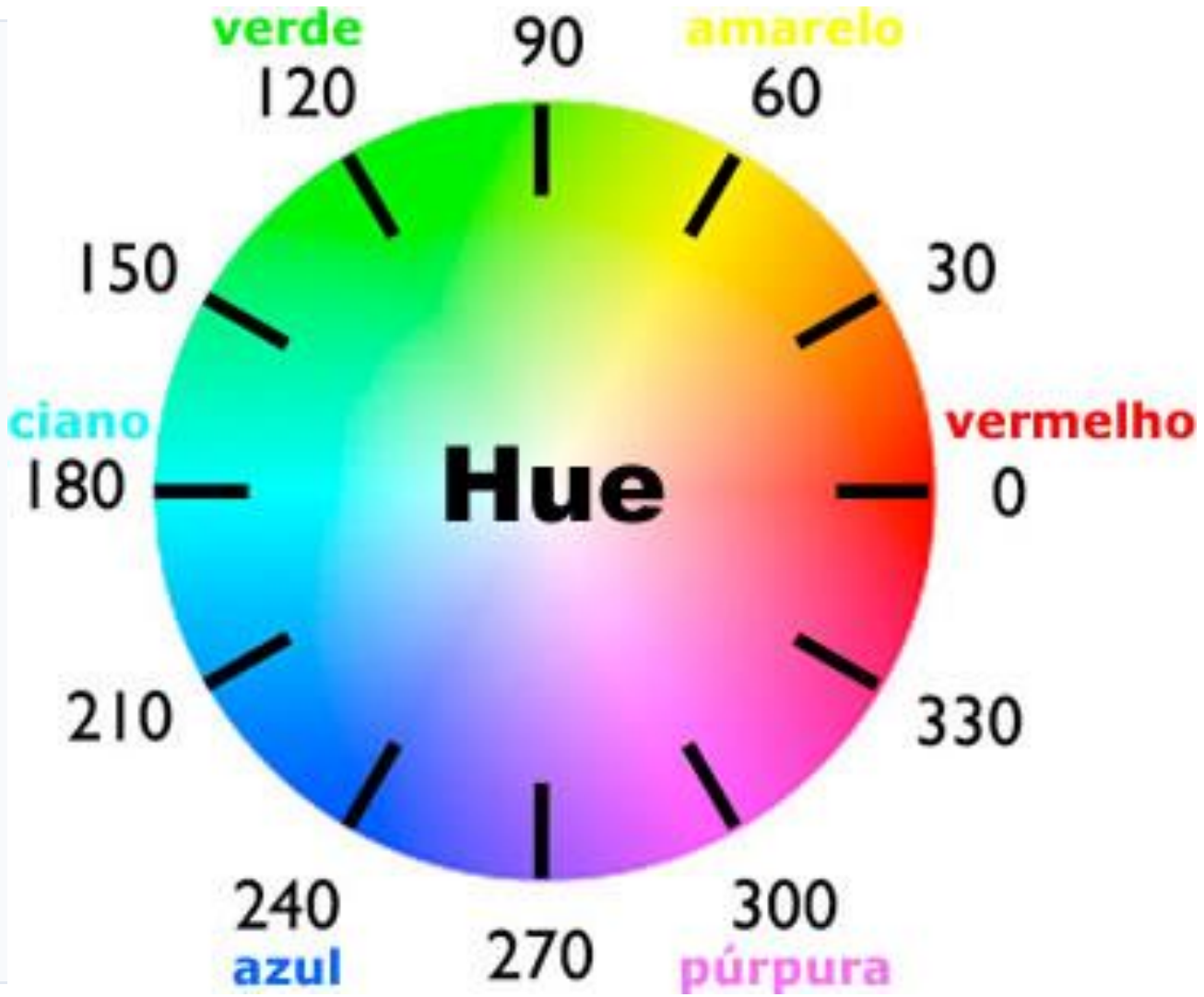
Fonte: https://www.researchgate.net/figure/Figura-412-O-espaco-de-cor-HSV-e-HSL-Retirado-de-wikipedia_fig7_345001282

Fundamentos de Cor

Geometria Cilíndrica: HSV e HSL

- **Hue (Matiz - H):** É a "cor" em si (o comprimento de onda dominante). É representado como um ângulo num círculo cromático, de 0º a 360º

Exemplo: 0º é **vermelho**, 120º é **verde**, 240º é **azul**).

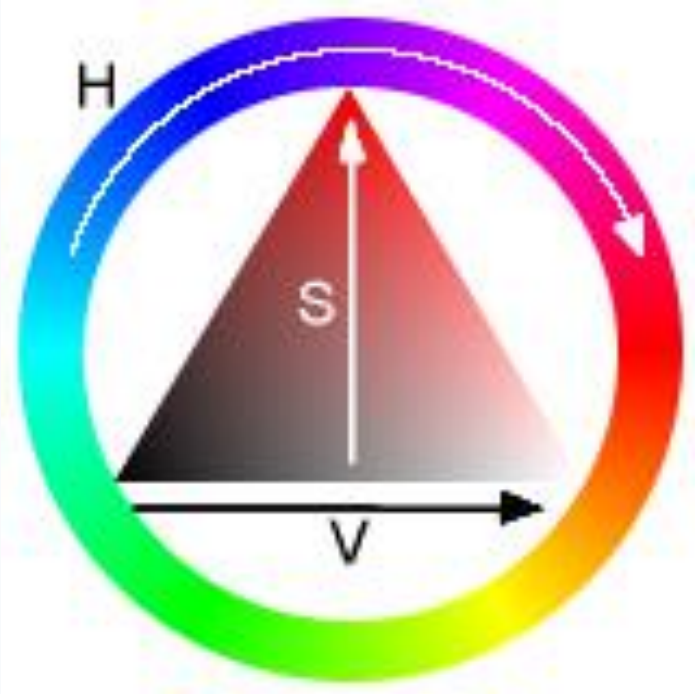


Fonte: https://maujor.com/tutorial/css3-modulo-para-cores.php

Fundamentos de Cor

Geometria Cilíndrica: HSV e HSL

- **Saturation (Saturação - S)**: Representa a pureza da cor, variando de 0 (escala de cinza/branco) a 1 (ou 100 %, cor pura, sem mistura de branco).



Fonte: https://pt.wikipedia.org/wiki/HSV

Fundamentos de Cor

Geometria Cilíndrica: HSV e HSL

- **Value (Valor/Brilho - V) e Lightness (Luminosidade - L):** É aqui que os modelos divergem.
- **No HSV**, o Value mede o brilho total. Um V=100 % com S=100% é uma cor pura e vibrante.
- **No HSL**, a Lightness mede a quantidade de branco/preto misturado. Se L=100%, a cor é totalmente branca, não importa o Matiz. A cor vibrante pura só ocorre quando L=50%.

Fundamentos de Cor

O Canal Alpha (RGBA)

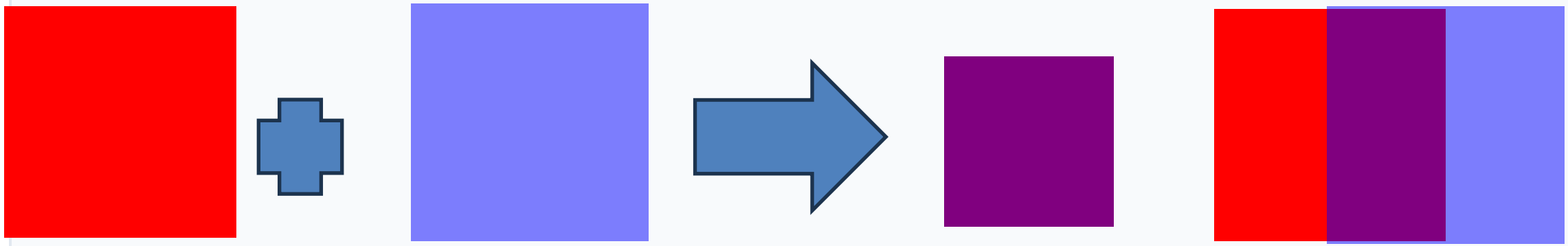
- Na década de 1980, Thomas Porter e Tom Duff formalizaram a álgebra de composição de imagens (Alpha Compositing).
- O canal Alpha (α) varia de 0.0 (totalmente transparente) a 1.0 (totalmente opaco).
- Quando sobrepomos um pixel de Primeiro Plano (Foreground - F) sobre um pixel de Fundo (Background - B), a cor resultante (C_{out}) obedece à seguinte equação de interpolação linear:

$$C_{out} = \alpha * C_F + (1 - \alpha) * C_B$$

Fundamentos de Cor

O Canal Alpha (RGBA)

- Vamos implementar isso em Python.
- A biblioteca nativa do Python **colorsys** nos ajuda a converter entre os diversos sistemas e RGB.
- Já o **Pillow** aplica a matemática do Canal Alpha.
- exemplo_canal_alfa.py



Fundamentos de Cor

O Canal Alpha (RGBA)

```
1    from PIL import Image
2
```

Fundamentos de Cor

O Canal Alpha (RGBA)

```
3  def criar_composicao_alpha():
4      print("\n--- Composição com Canal Alpha (RGBA) ---")
5      largura, altura = 400, 400
6
7      # Fundo (Background) - Vermelho totalmente Opaco (Alpha=255)
8      # RGBA = (R, G, B, Alpha)
9      fundo = Image.new("RGBA", (largura, altura), (255, 0, 0, 255))
10     fundo.save("fundo_alpha.png")
11     print("[Sucesso] Imagem 'fundo_alpha.png' salva.")
12
```

Fundamentos de Cor

O Canal Alpha (RGBA)

```
12
13      # Primeiro Plano (Foreground) - Azul 50%
14      # Alpha = 127 (Aproximadamente 50% de 255)
15      primeiro_plano = Image.new("RGBA", (largura, altura), (0, 0, 255, 127))
16      primeiro_plano.save("primeiro_plano_alpha.png")
17      print("[Sucesso] Imagem 'primeiro_plano_alpha.png' salva.")
18
```

Fundamentos de Cor

O Canal Alpha (RGBA)

```
18
19      # Aplica a equação de Porter-Duff
20      resultado = Image.alpha_composite(fundo, primeiro_plano)
21      resultado.save("resultado_alpha.png")
22      print("[Sucesso] Imagem 'resultado_alpha.png' salva.")
23
24  if __name__ == "__main__":
25      criar_composicao_alpha()
26
```

Fundamentos de Cor

HSV e HSL

Em Visão Computacional:

- Quase nunca se rastreia objetos usando RGB, porque o RGB varia com a iluminação do ambiente (uma sombra muda drasticamente o R, o G e o B).
- O modelo HSV é o mais utilizado, pois se a iluminação mudar, apenas o canal V (Brilho) sofre alteração. O canal H (Matiz) permanece intacto, permitindo que algoritmos de Inteligência Artificial reconheçam, por exemplo, placas de trânsito em dias nublados ou ensolarados com a mesma precisão.

Fundamentos de Cor

HSV e HSL

Em Interface e UX:

- Aplicações Web utilizam o padrão HSL(210, 50%, 50%) no CSS, pois permite criar versões "claras" e "escuras" (Dark Mode) de botões apenas alterando a variável L matematicamente.

Fundamentos de Cor

Criando imagens com HSV e HSL

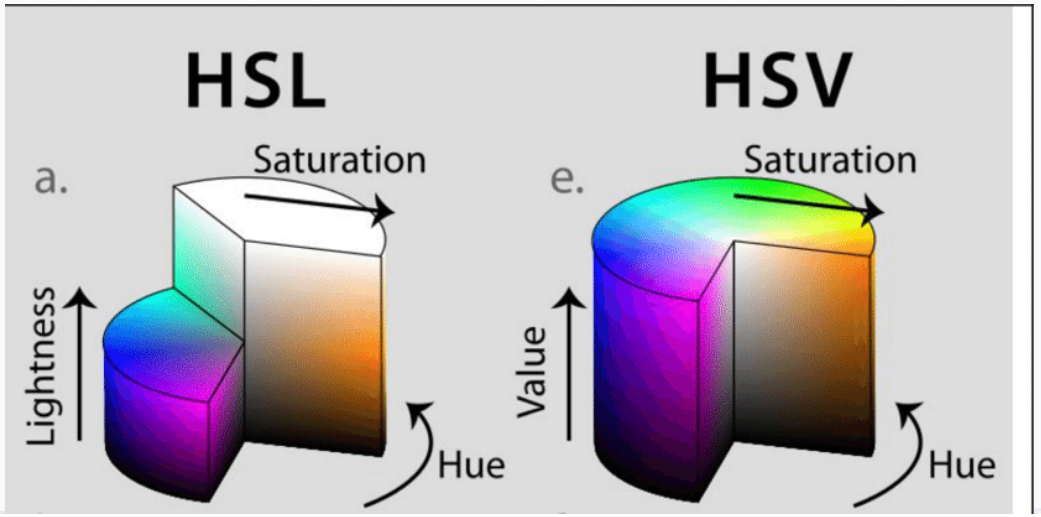
- A biblioteca **Pillow** opera primariamente nos espaços RGB, RGBA, CMYK e L (escala de cinza).
- Embora o Pillow possua algumas funcionalidades internas para HSV, a prática padrão é fazer a conversão dos parâmetros perceptuais (HSV/HSL) de volta para RGB antes de enviar os dados para o Framebuffer.

Fundamentos de Cor

Criando imagens com HSV e HSL

Mapeamento de domínios matemáticos (escalas):

- **Ângulos (Matiz/Hue)**: Na geometria, um círculo vai de 0º a 360º.
- No Python (colorsys), é preciso normalizar isso dividindo por 360.0, transformando em um espaço de [0.0, 1.0].



Fundamentos de Cor

Criando imagens com HSV e HSL

Mapeamento de domínios matemáticos (escalas):

- **Porcentagens (Saturação, Brilho, Luminosidade):** Normalmente pensamos em 0% a 100%.
- No código, transformamos em floats no intervalo [0.0, 1.0].

Fundamentos de Cor

Criando imagens com HSV e HSL

Mapeamento de domínios matemáticos (escalas):

- **Hardware de Vídeo (RGB):** Exige bytes inteiros.
- Portanto, a saída do colorsys (que é de 0.0 a 1.0) deve ser multiplicada por 255 e convertida para um tipo inteiro (int).

Fundamentos de Cor

Criando imagens com HSV e HSL

Vamos lá (exemplo_hsv_hsl.py)

```
1   import colorsys
2   from PIL import Image
3
```

Fundamentos de Cor

Criando imagens com HSV e HSL

Vamos lá (exemplo_hsv_hsl.py)

```
4   def criar_imagem_via_hsv(largura, altura, h_graus, s_pct, v_pct, nome_arquivo):
5       """
6       Cria uma imagem baseada no espaço HSV.
7       :param h_graus: Matiz (Hue) de 0 a 360 graus.
8       :param s_pct: Saturação (Saturation) de 0 a 100%.
9       :param v_pct: Valor/Brilho (Value) de 0 a 100%.
10      """
11      # Normalização do domínio físico para o domínio matemático [0.0, 1.0]
12      h_norm = h_graus / 360.0
13      s_norm = s_pct / 100.0
14      v_norm = v_pct / 100.0
```

Fundamentos de Cor

Criando imagens com HSV e HSL

```
15
16      # Transformação do espaço cilíndrico (HSV) para vetorial (RGB)
17      r_float, g_float, b_float = colorsys.hsv_to_rgb(h_norm, s_norm, v_norm)
18
19      # Converte de ponto flutuante para inteiros de 8 bits (0-255)
20      r = int(r_float * 255)
21      g = int(g_float * 255)
22      b = int(b_float * 255)
23
24      imagem = Image.new("RGB", (largura, altura), (r, g, b))
25      imagem.save(nome_arquivo)
26      print(f"[HSV] Imagem salva: {nome_arquivo} | RGB final: ({r}, {g}, {b})")
```

Fundamentos de Cor

Criando imagens com HSV e HSL

Vamos lá (exemplo_hsv_hsl.py)

```
28  def criar_imagem_via_hsl(largura, altura, h_graus, s_pct, l_pct, nome_arquivo):
29      """
30      Cria uma imagem baseada no espaço HSL.
31      :param h_graus: Matiz (Hue) de 0 a 360 graus.
32      :param s_pct: Saturação (Saturation) de 0 a 100%.
33      :param l_pct: Luminosidade (Lightness) de 0 a 100%.
34      """
35      # Mesma coisa, faz a normalização para o intervalo [0.0, 1.0]
36      h_norm = h_graus / 360.0
37      s_norm = s_pct / 100.0
38      l_norm = l_pct / 100.0
```

Fundamentos de Cor

Criando imagens com HSV e HSL

```
    # tbm tranforma o espaço cilíndrico (HLS/HSL) para vetorial (RGB)
    r_float, g_float, b_float = colorsys.hls_to_rgb(h_norm, l_norm, s_norm)

    # Converte para inteiros de 8 bits
    r = int(r_float * 255)
    g = int(g_float * 255)
    b = int(b_float * 255)

    imagem = Image.new("RGB", (largura, altura), (r, g, b))
    imagem.save(nome_arquivo)
    print(f"[HSL] Imagem salva: {nome_arquivo} | RGB final: ({r}, {g}, {b})")
```

Fundamentos de Cor

```
70  if __name__ == "__main__":
71      LARGURA = 400
72      ALTURA = 400
73
74      # Ciano Vibrante em HSV - H = 180 (Ciano), S = 100% (Puro), V = 100% (Brilho máximo)
75      criar_imagem_via_hsv(LARGURA, ALTURA, 180, 100, 100, "ciano_hsv.png")
76
77      # Ciano Vibrante em HSL - Se liga nessa dica:
78      # Para obter a cor pura em HSL, a Luminosidade (L) deve ser exatos 50%.
79      # Se for 100%, será totalmente branco.
80      criar_imagem_via_hsl(LARGURA, ALTURA, 180, 100, 50, "ciano_hsl.png")
81
82      # Efeito desbotado Desbotado ("Pastel") usando HSL
83      # Ciano puro (H=180, S=100%), e aumenta a luminosidade para 80%
84      # Isso mistura "tinta branca" à cor matematicamente.
85      criar_imagem_via_hsl(LARGURA, ALTURA, 180, 100, 80, "ciano_pastel_hsl.png")
```

Dúvidas e Discussão

Este é o momento para aprofundarmos nos conceitos.

raulivan@cefetmg.br

SIGAA

Vamos praticar

Desafio do Chroma Key

O objetivo do Chroma Key é identificar um fundo de cor sólida (geralmente verde ou azul) e torná-lo transparente (Alpha = 0), permitindo que outra imagem seja colocada no fundo.



Vamos praticar

Desafio do Chroma Key

Por que usamos o verde?

As câmeras digitais modernas possuem um arranjo de sensores chamado Filtro de Bayer, que contém o dobro de fotorreceptores verdes em relação aos azuis e vermelhos.

Logo, o canal verde possui menos ruído e maior definição.

Vamos praticar

Desafio do Chroma Key

Imagine um estúdio com um fundo verde.
Devido à iluminação, o centro do fundo estará muito claro (verde limão), e os cantos terão sombras (verde musgo escuro).
No cubo 3D do sistema RGB, esses dois tons de verde estão em coordenadas completamente distantes.
(RGB) Se você fizer um if (R == 0 and G == 255 and B == 0), seu código só removerá um pixel perfeito, deixando todo o resto do fundo intacto.

Vamos praticar

Desafio do Chroma Key

A solução arquitetural é mudar o nosso sistema de coordenadas para o Cilindro HSV.
No espaço HSV, tanto o "verde claro" quanto o "verde musgo escuro" possuem a mesma raiz angular (o Matiz / Hue), variando apenas na saturação ou no brilho.

Vamos praticar

Desafio do Chroma Key

A lógica matemática:

O verde puro encontra-se no ângulo de 120º.

Pode definir uma janela de tolerância, por exemplo, de 90º a 150º.

Normalizando para o intervalo matemático [0.0, 1.0], dividimos por 360, obtendo um intervalo de Matiz H ϵ [0.25, 0.41].

Ignoramos pixels que são brancos/cinzas demais (Saturação muito baixa) ou pretos demais (Brilho muito baixo).

Vamos praticar

Desafio do Chroma Key

Implemente um algoritmo que aplique chroma key em uma imagem.

Deve usar as Libs colorsys, Image (PIL) e ImageDraw (PIL)





Próxima Aula

Transformações Gráficas 2D

